

Documentație

Partiționarea în clase de echivalență și analiza valorilor de frontieră

Framework

Pentru partiționarea în clase de echivalență și analiza valorilor de frontieră, am folosit framework-ul de testare Pytest.

Comandă de rulare a testelor:

```
python -m pytest tests/equivalence/test_equivalence.py -v
```

Metode testate

Am testat toate cazurile corespunzătoare claselor de echivalență ale metodelor principale din clasa SistemRezervareAvion, rezerva_loc, anuleaza_rezervare și este_loc_disponibil.

1. Rezerva_loc

Pentru fiecare parametru al funcției rezerva_loc(rand, litera_loc, varsta_pasager, are_bagaj_cala) am creat următoarele clase de echivalență:

- rand (int) – notez cu r

R_1 = {1, 2} => rând valid cu preț de Business Class

R_2 = 3...10 => rând valid cu preț de Economy Class

R_3 = {r < 1 || r > 10} => rând invalid => ValueError

R_4 = r este alt tip de date => rând invalid => TypeError

R_5 = r este bool => rând invalid => TypeError

- analiza de frontiera:

- rand valid

- limita inferioara: r = 1
- limita superioara: r = 10

- rand invalid

- limita inferioara: r = 0
- limita superioara: r = 11

- litera_loc (str) – notez cu l

L_1 = {'A', 'B', 'C', 'D', 'E', 'F'} => literă mare validă

L_2 = {'a', 'b', 'c', 'd', 'e', 'f'} => literă mică validă

L_3 = l este str, dar nu aparține intervalului A-F sau a-f => literă invalidă => ValueError

L_4 = l este alt tip de date => literă invalidă => ValueError

L_5 = l este str, dar are lungime diferită de 1 => literă invalidă => ValueError

- analiza de frontiera:
 - litera_loc valid
 - limita inferioara: l = 'A' sau l = 'a'
 - limita superioara: l = 'F' sau l = 'f'
 - litera_loc invalid
 - limita superioara: l = 'G' sau l = 'g'

- varsta_pasager (int)

V_1 = {0, 1} => vârstă validă pentru infant, preț redus 90%

V_2 = {2...12} => vârstă validă pentru copil, preț redus 50%

V_3 = {13...59} => vârstă validă pentru adult, preț întreg

V_4 = {v >= 60} => vârstă validă pentru senior, preț redus 50%

V_5 = {v < 0} => vârstă invalidă => ValueError

V_6 = v este alt tip de date => vârstă invalidă => TypeError

V_7 = v este bool => vârstă invalidă => TypeError

- analiza de frontiera:
 - varsta_pasager valid pentru infant
 - limita inferioara: v = 0
 - limita superioara: v = 1
 - varsta_pasager valid pentru copil
 - limita inferioara: v = 2
 - limita superioara: v = 12
 - varsta_pasager valid pentru adult
 - limita inferioara: v = 13
 - limita superioara: v = 59
 - varsta_pasager valid pentru senior
 - limita inferioara: v = 60
 - varsta_pasager invalid
 - limita inferioara: v = -1

- are_bagaj_cala (bool)

B_1 = b este True și pasagerul nu este infant => bagaj de cală inclus, se adaugă 20 la preț

B_2 = b este False => bagaj de cală neinclus, prețul nu se modifică

B_3 = b este True și pasagerul este infant => bagajul de cală nu modifică prețul

B_4 = b este alt tip de date => bagaj invalid => TypeError

2. Anuleaza_rezervare

Pentru fiecare parametru al funcției `anuleaza_rezervare(rand, litera_loc)` am creat următoarele clase de echivalență:

- `rand`

$R_1 = \{1, 2\} \Rightarrow$ rând valid de Business Class

$R_2 = 3 \dots 10 \Rightarrow$ rând valid de Economy Class

$R_3 = \{r < 1 \parallel r > 10\} \Rightarrow$ rând invalid $\Rightarrow$ `ValueError`

$R_4 = r$ este alt tip de date $\Rightarrow$ rând invalid $\Rightarrow$ `TypeError`

$R_5 = r$ este bool $\Rightarrow$ rând invalid $\Rightarrow$ `TypeError`

$R_6 = r$ este valid, dar locul este deja liber $\Rightarrow$ anularea returnează `False`

- analiza de frontiera:

- `rand` valid

- limita inferioara: $r = 1$
- limita superioara: $r = 10$

- `rand` invalid

- limita inferioara: $r = 0$
- limita superioara: $r = 11$

- `litera_loc`: aceleași clase de echivalență și frontiere ca în metoda `rezerva_loc`

3. Este_loc_disponibil

Pentru fiecare parametru al funcției `este_loc_disponibil(rand, litera_loc)` am creat următoarele clase de echivalență.:

- `rand`

$R_1 = r$ este valid și locul este liber $\Rightarrow$ metoda returnează `True`

$R_2 = r$ este valid și locul este ocupat $\Rightarrow$ metoda returnează `False`

$R_3 = \{r < 1 \parallel r > 10\} \Rightarrow$ rând invalid $\Rightarrow$ `ValueError`

- analiza de frontiera:

- `rand` valid

- limita inferioara: $r = 1$
- limita superioara: $r = 10$

- `rand` invalid

- limita inferioara: $r = 0$
- limita superioara: $r = 11$

- litera_loc: aceleași clase de echivalență și frontiere ca în metoda rezerva_loc

Raport AI

Am folosit ChatGPT Plus pentru a evidenția diferența dintre testele scrise manual și cele generate cu ajutorul unui tool AI.

Prompt:

“Creează teste bazate pe partiționarea în clase de echivalență și analiza valorilor de frontieră, folosind pytest, pentru metodele rezerva_loc, anuleaza_rezervare și este_loc_disponibil din clasa SistemRezervareAvion.”

Am atașat și codul clasei în prompt.

Răspuns:

“Voi construi un fișier de teste pytest orientat pe clase de echivalență și valori de frontieră: validări de input, limite de rând/literă/vârstă, prețuri la frontiere, loc ocupat, dezechilibru și anulare/disponibilitate.

Mai jos ai un exemplu de fișier test_sistem_rezervare_avion.py. Înlocuiește from sistem_rezervare_avion import SistemRezervareAvion cu importul corect pentru proiectul tău.”

Am copiat codul primit în răspuns și l-am introdus în proiect, în fișierul “test_equivalenceAI.py”.

Creează teste bazate pe partiționarea în clase de echivalență și analiza valorilor de frontieră, folosind pytest, pentru metodele rezerva_loc, anuleaza_rezervare și este_loc_disponibil din clasa:

```
class SistemRezervareAvion:
```

```
    """
```

```
    Sistem de rezervare locuri avion:
```

```
    Avion cu 10 randuri x 6 locuri (A, B, C, D, E, F)
```

```
    Randurile 1-2 sunt Business Class (+50 RON)
```

Show more ▾

Voi construi un fișier de teste pytest orientat pe clase de echivalență și valori de frontieră: validări de input, limite de rând/literă/vârstă, prețuri la frontiere, loc ocupat, dezechilibru și anulare/disponibilitate.

Thought for a couple of seconds >

Mai jos ai un exemplu de fișier `test_sistem_rezerva_vion.py`. Înlocuiește `from sistem_rezervare_avion import SistemRezervareAvion` cu importul corect pentru proiectul tău.

+ Ask anything

Instant ▾



Thought for a couple of seconds >

Mai jos ai un exemplu de fișier `test_sistem_rezervare_avion.py`. Înlocuiește `from sistem_rezervare_avion` `import SistemRezervareAvion` cu importul corect pentru proiectul tău.

```
Python

import pytest

from sistem_rezervare_avion import SistemRezervareAvion

@pytest.fixture
def sistem():
    return SistemRezervareAvion()

# =====
# rezerva_loc - clase de echivalenta + valori de frontiera
# =====

@pytest.mark.parametrize(
    "rand, litera, varsta, bagaj, pret_asteptat",
    [
        # randuri business: frontiera 1-2
        (1, "A", 30, False, 150.0),
        (2, "D", 30, False, 150.0),
```

Am rulat testele generate și au trecut toate.

```
test_equivalence.py  test_equivalenceAI.py U X

15
16 # =====
17 # rezerva_loc - clase de echivalenta + valori de frontiera
18 # =====
19
20 @pytest.mark.parametrize(
21     "rand, litera, varsta, bagaj, pret_asteptat",
22     [
23         # randuri business: frontiera 1-2
24         (1, "A", 30, False, 150.0),
25         (2, "D", 30, False, 150.0),
26
27         # rand economy: frontiera imediat dupa business
28         (3, "A", 30, False, 100.0),
29
30         # litere valide, inclusiv lowercase
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\ana\Documents\facultate\ANUL III\sem 2\tss\Proiect_TestareUnitarainPython> python -m pytest tests/equivalence/test_equivalenceAI.py -v
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.3, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\ana\Documents\facultate\ANUL III\sem 2\tss\Proiect_TestareUnitarainPython
collected 66 items

tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[1-A-30-False-150.0] PASSED [ 1%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[2-D-30-False-150.0] PASSED [ 3%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[3-A-30-False-100.0] PASSED [ 4%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[10-f-30-False-100.0] PASSED [ 6%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[3-B-0-False-10.0] PASSED [ 7%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[3-C-1-True-10.0] PASSED [ 9%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[4-A-2-True-70.0] PASSED [ 10%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[4-B-12-False-50.0] PASSED [ 12%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[4-C-13-True-120.0] PASSED [ 13%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[5-A-59-False-100.0] PASSED [ 15%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_valid_preturi_frontiera[5-B-60-True-70.0] PASSED [ 16%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_rand_invalid_valoare[0] PASSED [ 18%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_rand_invalid_valoare[11] PASSED [ 19%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_rand_invalid_valoare[-1] PASSED [ 21%]
tests/equivalence/test_equivalenceAI.py::test_rezerva_loc_rand_invalid_tip[1.5] PASSED [ 22%]
```

